

```

369     )
370     return {**trip, "mass_kg": mass, "volume_m3": volume, "duration_s": duration}
371
372
373 def exact_q1_service(
374     dem: DemGrid,
375     depot: Node,
376     service: Node,
377     boxes: pd.DataFrame,
378     types: dict[str, TransportType],
379 ) -> list[dict[str, Any]]:
380     boxes = boxes.reset_index(drop=True)
381     n = len(boxes)
382     full = (1 << n) - 1
383     candidates: dict[int, list[dict[str, Any]]] = {}
384     for mask in range(1, full + 1):
385         idx = [i for i in range(n) if mask & (1 << i)]
386         batch = boxes.iloc[idx]
387         for type_id, g in types.items():
388             metrics = batch_metrics(dem, depot, service, g, batch)
389             feasible = (
390                 metrics["mass_kg"] <= g.payload_kg + 1e-9
391                 and metrics["volume_m3"] <= g.volume_m3 + 1e-12
392                 and metrics["energy_kwh"] <= (1.0 - g.reserve) * g.energy_kwh + 1e-9
393             )
394             if feasible:
395                 candidates.setdefault(mask, []).append(
396                     {
397                         "mask": mask,
398                         "type_id": type_id,
399                         "box_ids": list(batch["货箱编号"].astype(str)),
400                         **metrics,
401                     }
402                 )
403     dp: dict[int, tuple[tuple[float, float, float], list[dict[str, Any]]]] = {0: ((0, 0.0, 0.0), [])}
404     for covered in range(1, full + 1):
405         if covered not in dp:
406             continue
407         remaining = full ^ covered
408         if remaining == 0:
409             continue
410         first_bit = remaining & -remaining
411         for mask, options in candidates.items():
412             if not (mask & first_bit) or (mask & covered):
413                 continue
414             for cand in options:
415                 old_cost, old_path = dp[covered]
416                 new_cost = (
417                     old_cost[0] + 1,
418                     old_cost[1] + cand["energy_kwh"],
419                     old_cost[2] + cand["duration_s"],
420                 )
421                 new_mask = covered | mask
422                 if new_mask not in dp or new_cost < dp[new_mask][0]:
423                     dp[new_mask] = (new_cost, old_path + [cand])
424     if full not in dp:
425         raise RuntimeError(f"Q1 service {service.node_id} has no feasible partition")
426     return dp[full][1]

```

C.2 问题二：多点路线递推与双资源排程

代码清单3展示多点路线的统一事件递推。程序按访问顺序逐段计算剩余载荷下的飞行能耗，在每个服务区按硬截止、优先系数和货箱编号安排交接，并返回逐箱完成时刻、返航 SOC 和完整阶段记录。

Listing 3 多点运输路线与逐箱交付时间轴

```

1329 def route_timeline(
1330     dem: DemGrid,
1331     depot: Node,
1332     nodes: dict[str, Node],
1333     g: TransportType,

```